



MODULE 2

LLMs as Reasoning Engines

How Large Language Models Think, Reason, and Generate



SESSION ROADMAP



1. FUNDAMENTALS OF LARGE LANGUAGE MODELS

Tokenization, context windows, next-token prediction, emergent reasoning



2. PROMPT ENGINEERING TECHNIQUES

Zero-shot, few-shot, role-based prompting, system instructions



3. REASONING TECHNIQUES IN LLMs

Chain-of-Thought, self-consistency, reflection, structured outputs



4. LIMITATIONS OF LLMs

Hallucinations, knowledge gaps, context limits, reliability

SECTION 1

Fundamentals of Large Language Models

How LLMs process text, predict tokens, and develop reasoning

What is a Large Language Model?

A Large Language Model (LLM) is a neural network trained on massive text corpora to predict the next token in a sequence. Through scale and training, LLMs develop emergent capabilities: reasoning, planning, code generation, and more.



Scale

Billions of parameters
trained on trillions of tokens



Prediction

Generates text one
token at a time



Emergence

Capabilities not explicitly
programmed but learned



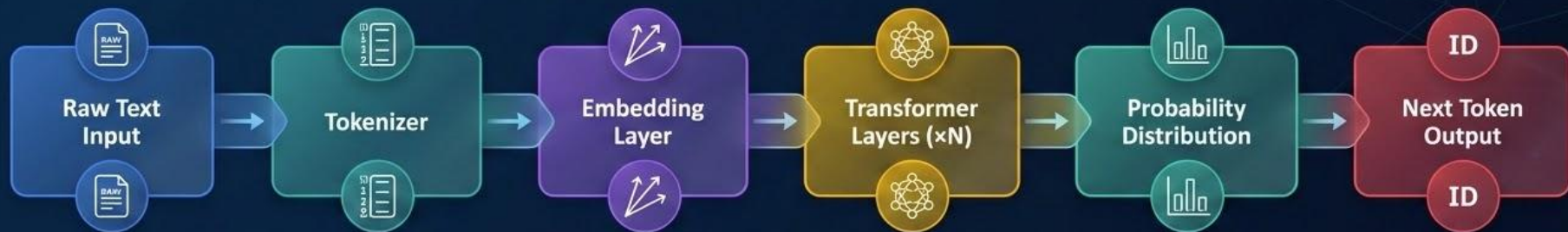
Versatility

One model handles many
tasks across domains

The Modern LLM Landscape

Model Family	Developer	Key Characteristics
GPT-4 / GPT-4o	OpenAI	Strong reasoning, multimodal, function calling
Claude 3.5 / 4	Anthropic	Long context (200K), safety-focused, tool use
Gemini 2.0	Google	Multimodal native, code, 1M+ context
Llama 3.1	Meta	Open-weight, community fine-tuning, 405B
Mistral Large	Mistral AI	Efficient, multilingual, open-weight options
Command R+	Cohere	RAG-optimized, enterprise, grounded generation
Deepseek R1	Deepseek	Reasoning-specialized, chain-of-thought native

How an LLM Generates Text



Tokenization



"Hello world" → [15496, 995]

Text is split into subword tokens — each gets a numeric ID. Vocabulary typically 50K-100K tokens.

Embeddings



Each token ID maps to a high-dimensional vector (e.g., 4096 dims). These vectors encode semantic meaning — similar words have similar vectors.

Transformer Layers



Self-attention mechanism allows every token to "attend" to every other token. This captures relationships, context, and long-range dependencies.

Tokenization: Text → Numbers



Before an LLM can process text, it must convert words into numeric tokens. This process fundamentally shapes what the model can "see."

Tokenization Example:

"Artificial intelligence is amazing"

Subword Token List

[Art', 'ificial', ' intelligence', ' is', ' amazing']



Numeric Token ID List [15836, 14803, 11478, 318, 4998]



Subword Tokenization:

Words are split into common subword units — "playing" → ["play", "ing"]. This handles unseen words and reduces vocabulary size.



Token ≠ Word:

On average, 1 English word ≈ 1.3 tokens. Code and non-English text typically use more tokens per word.



BPE (Byte-Pair Encoding):

The most common algorithm. Iteratively merges frequent character pairs into tokens. Used by GPT, Claude, and most modern LLMs.



Special Tokens:

Models use markers like [BOS] (beginning), [EOS] (end), [PAD] (padding) to structure sequences.

Embeddings & Positional Encoding



Token Embeddings

Each token ID maps to a learned dense vector of dimension d_{model} (e.g., 4096 for LLaMA-7B). This embedding table is a trainable weight matrix W_E of shape $(V \times d)$.

"cat" $\rightarrow$ [

0.3	-0.7	0.1	0.8	-0.4	0.5
1	7	2	5	4	6

 , ...]



Positional Encoding

Transformers have no inherent notion of order. Positional encodings inject sequence position info.

Combined Input Representation

$$x = \text{TokenEmbedding}(\text{token_id}) + \text{PositionalEncoding}(\text{position})$$

The resulting vector $x \in \mathbb{R}^{d_{\text{model}}}$ is the input to the first Transformer layer. The embedding dimension d_{model} is a key hyperparameter: 768 (BERT-base), 4096 (LLaMA-7B), 12288 (GPT-3 175B).

Sequence to Sequence Learning with Neural Networks

Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com

Abstract

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT-14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous state of the art. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
 Jacobs University Bremen, Germany

kyungHyun Cho Yoshua Bengio*
 Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

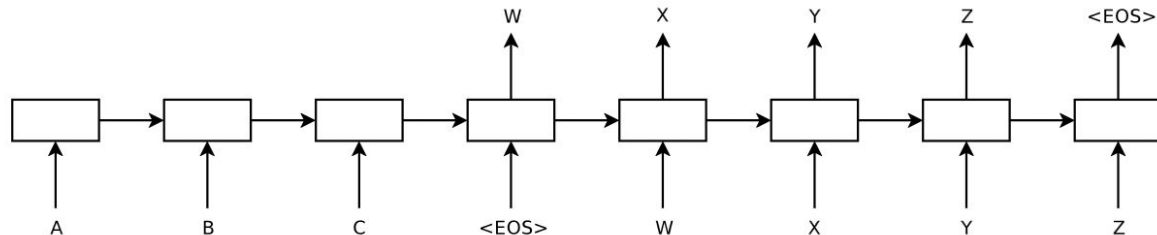
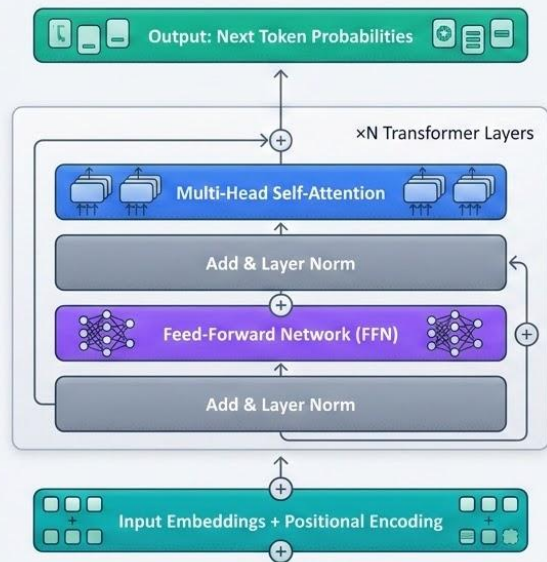


Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

The Transformer Architecture

Introduced in "Attention Is All You Need" (Vaswani et al., 2017)



Key Innovations



Self-Attention

Each token an attention accessed multi-head dependencies



Residual Connections

Skip connection with line and aw skip connection networks



Parallelizable

Unlike RNNs, all positions are processed simultaneously during training



Scalable

Performance scales predictably with model size, data, and compute (scaling laws)

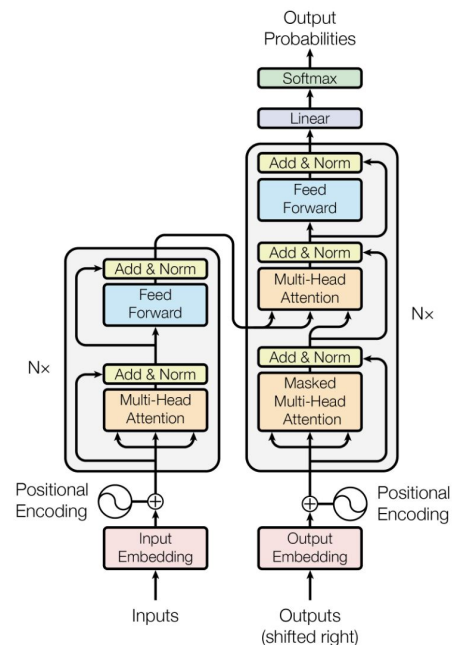


Figure 1: The Transformer - model architecture.

https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Self-Attention Mechanism

The core computation enabling contextual understanding

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

Where $Q = X \cdot W_Q$, $K = X \cdot W_K$, $V = X \cdot W_V$ are linear projections of the input



Query

"What am I looking for?" — each token's search vector



Key

"What do I contain?" — each token's descriptor vector



Value

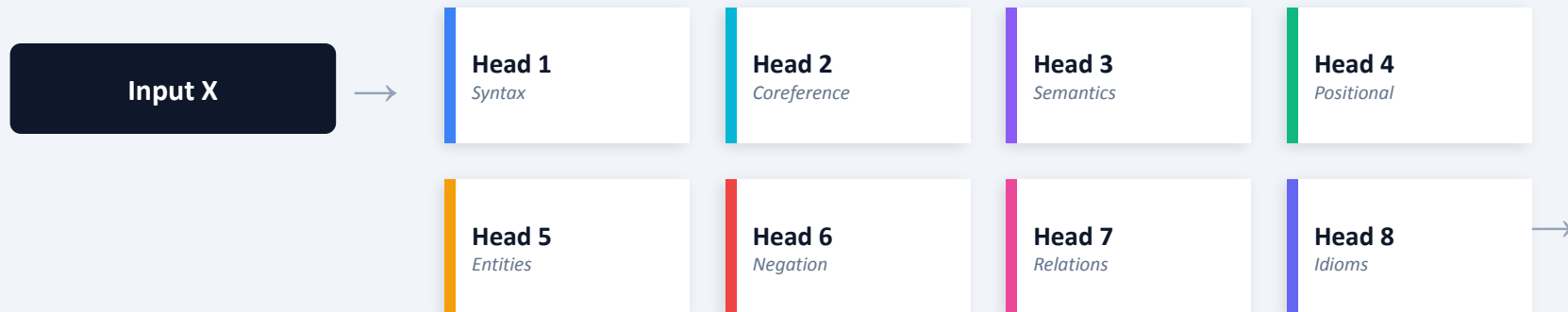
"What info do I carry?" — actual content to aggregate

Computation Steps

1. Project input X to Q , K , V via learned weight matrices
2. Compute dot products $Q \cdot K^T \rightarrow$ raw attention scores ($n \times n$ matrix)
3. Scale by $1/\sqrt{d_k}$ to prevent vanishing gradients in softmax
4. Apply causal mask (decoder-only: prevent attending to future tokens)
5. Softmax $\rightarrow$ attention weights; Multiply by $V \rightarrow$ contextualized output

Multi-Head Attention

Running multiple attention computations in parallel for richer representations



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O \quad \text{where} \quad \text{head}_i = \text{Attention}(Q \cdot W_{Q_i}, K \cdot W_{K_i}, V \cdot W_{V_i})$$

Why Multiple Heads?

Each head operates in a $d_k = d_{\text{model}}/h$ subspace. Different heads learn different linguistic patterns: syntactic dependencies, semantic roles, coreference, etc.

Typical Configurations

GPT-3: 96 heads, $d_{\text{model}}=12288$
LLaMA-7B: 32 heads, $d_{\text{model}}=4096$
LLaMA-70B: 64 heads + GQA (8 KV heads)

Pre-Training and Fine Tuning

Training Pipeline

1

Data Collection

Web crawl, books, code, wiki —
trillions of tokens

2

Data Cleaning

Dedup, filter toxicity, quality scoring,
domain mixing

3

Tokenize & Batch

Pack sequences to max_len, create
training batches

4

Gradient Descent

AdamW optimizer, cosine LR
schedule, thousands of GPUs

Fine-Tuning & Alignment

From raw text predictor to helpful, harmless, and honest assistant

Stage 1: Supervised Fine-Tuning (SFT)

- 1 Train on curated (prompt, response) pairs from human annotators. Teaches instruction-following and conversational format.

Stage 2: Reward Model Training

- 2 Human raters rank multiple model outputs. A reward model $R(x,y)$ is trained on these preferences using the Bradley-Terry model.

Next-Token Prediction Mechanism



The core operation of every LLM: given all previous tokens, predict the probability distribution over the next token.

Prediction Example:

Input: "The capital of France is"

Probability Distribution:

"Paris"	→	92.3%
"Lyon"	→	2.1%
"Berlin"	→	0.4%
"the"	→	1.8%

Greedy Decoding

Always pick the highest probability token. Deterministic but can be repetitive.

Temperature

Controls randomness: low (0.0) = deterministic, high (1.0+) = creative/random.

Top-p (Nucleus)

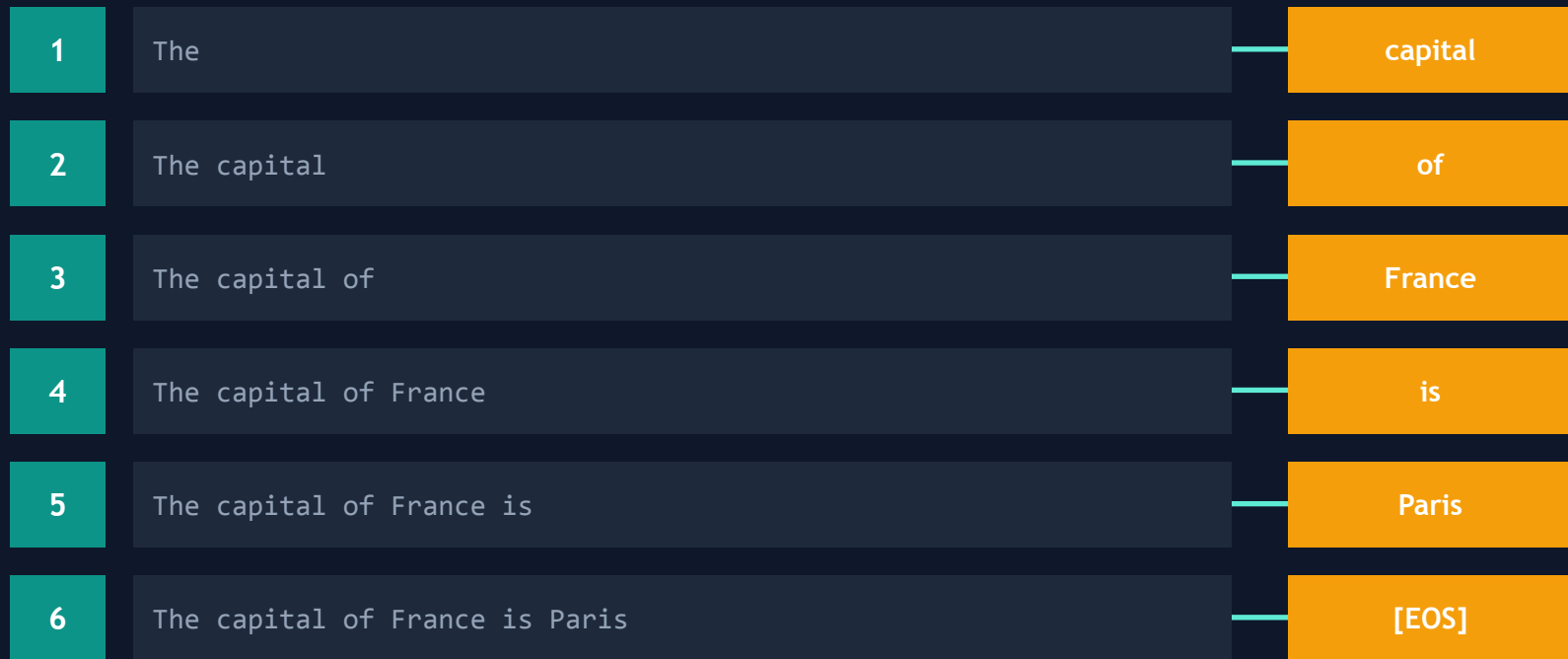
Sample from smallest set of tokens whose cumulative probability exceeds p (e.g. 0.9).

Top-k Sampling

Only consider the k most probable tokens. Filters out unlikely completions.

Inference: Autoregressive Generation

LLMs generate text one token at a time, feeding each output back as input for the next prediction.



Each step requires a full forward pass through all transformer layers — this is why generation is sequential and slower than processing input.

Context Windows: The Model's Memory



The context window is the maximum number of tokens the model can process in a single forward pass: it defines how much the model can "see" at once.

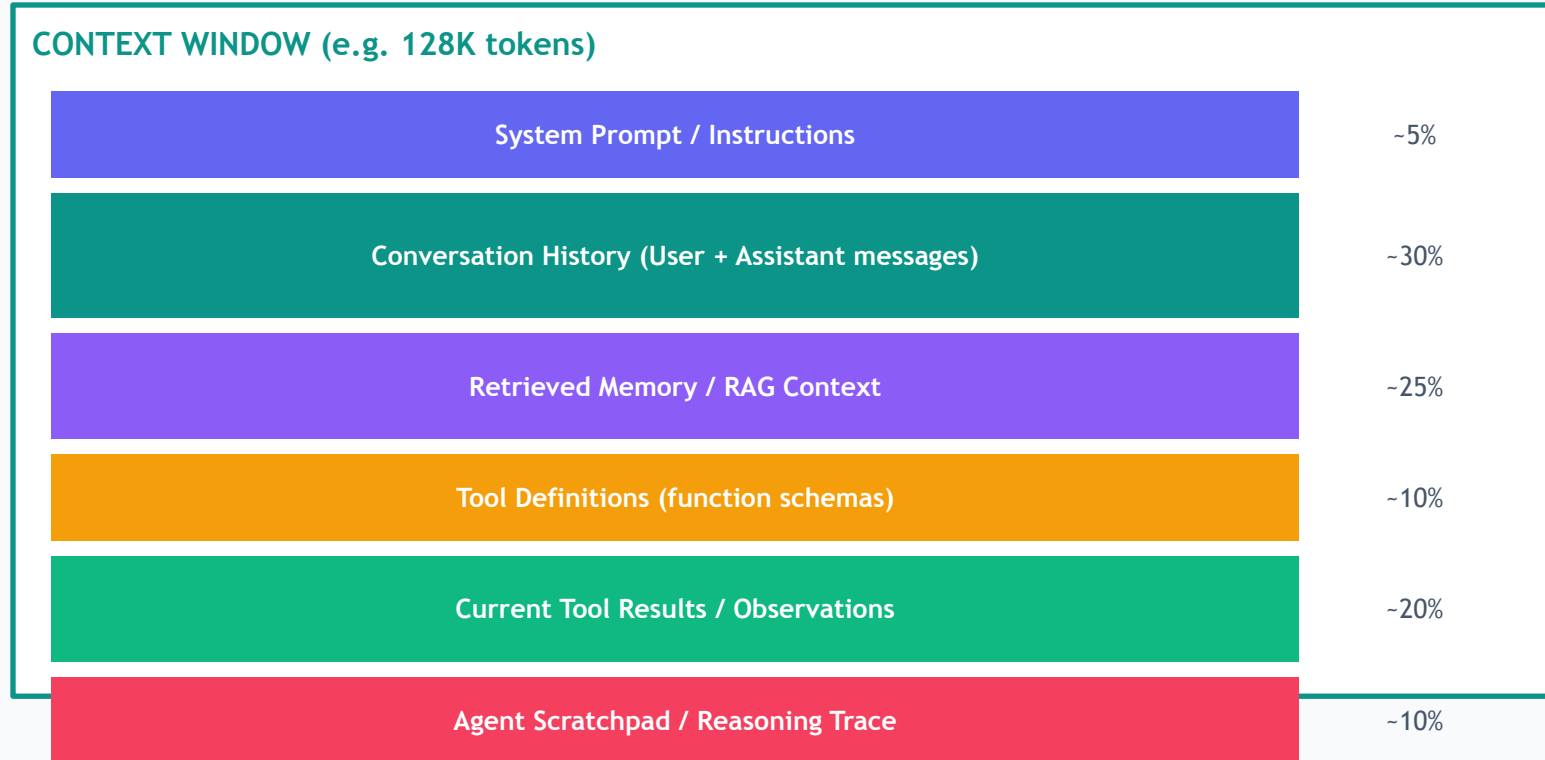
Model	Context Window	Approximate Pages
GPT-3.5	4,096 tokens	~6 pages
GPT-4	128K tokens	~200 pages
Claude 3.5 Sonnet	200K tokens	~300 pages
Gemini 2.0	1M+ tokens	~1,500 pages
Llama 3.1	128K tokens	~200 pages

Why Context Windows Matter for Agents:

The context window limits how much conversation history, tool results, and memory an agent can hold at once. When the window fills up, older information must be truncated or summarized, leading to potential information loss.

Context Window Architecture

What fills the context window in an agentic system:



Scaling Laws & Model Size

Performance improves predictably with scale: more parameters, more data, and more compute lead to better capabilities.

Parameters

7B → 70B → 405B → 1.8T

More nuanced reasoning, fewer factual errors, better instruction following

Training Data

1T → 15T tokens

Broader knowledge, better generalization, more languages and domains

Compute (FLOPs)

10^{22} → 10^{25}

Enables larger models and longer training: the primary cost driver

Context Length

2K → 128K → 1M+

Process longer documents, maintain coherence over extended conversations

SECTION 2

Prompt Engineering Techniques

Controlling LLM behavior through careful input design

What is Prompt Engineering?



Prompt engineering is the practice of designing and structuring inputs to guide LLM behavior, controlling output format, reasoning depth, tone, and accuracy without modifying the model itself.

Cost-Effective

No fine-tuning or retraining required: just change the text input

Portable

Same techniques work across different LLM providers

Immediate

Results are instant: iterate and test prompts in real-time

Powerful

Can dramatically improve accuracy, reduce hallucination, and control format

Zero-Shot Prompting

Ask the model to perform a task with no examples: relying entirely on its pre-trained knowledge and instruction following.

Prompt

Classify the following text as positive, negative, or neutral:

"The product arrived on time but the packaging was damaged."

Response

Neutral.

The review contains both a positive element (on-time delivery) and a negative element (damaged packaging), making the overall sentiment mixed/neutral.

When to Use:

Simple classification, translation, summarization, or tasks where the model's pre-trained knowledge is sufficient. Works best with clear, specific instructions.

Few-Shot Prompting

Provide 2-5 examples of input-output pairs before the actual task: teaching the model the pattern by demonstration.

Few-Shot Prompt Example

Extract the product and sentiment:

Text: "Love my new AirPods!" → Product: AirPods, Sentiment: Positive

Text: "The Galaxy S24 battery is terrible" → Product: Galaxy S24, Sentiment: Negative

Text: "MacBook Pro is okay for the price" → Product: MacBook Pro, Sentiment: Neutral

Text: "The Pixel 9 camera blew me away"
→ Product: Pixel 9, Sentiment: Positive

↑ Examples (demonstrations)

↑ Actual task + model's output

Role-Based Prompting



Assign the model a persona or role to shape its tone, expertise level, and behavioral patterns.

Expert Data Scientist

"You are a senior data scientist at a FAANG company. Analyze this dataset and recommend the best ML approach..."

Technical depth, statistical rigor, industry best practices

Teacher

"You are a friendly teacher explaining concepts to a 10-year-old. Use simple language and fun analogies..."

Simple vocabulary, analogies, encouragement, step-by-step

Code Reviewer

"You are a strict code reviewer. Find bugs, security issues, and performance problems. Be thorough..."

Critical analysis, specific line references, security focus

Legal Advisor

"You are a corporate attorney specializing in contract law. Review this agreement for risks..."

Formal language, clause-by-clause analysis, risk flags

System Instructions



System messages set the foundational behavior, constraints, and personality of the model, they persist across the entire conversation.

System Message Anatomy:

Role Definition

You are a helpful AI assistant specializing in financial analysis.

Behavioral Rules

Always cite sources. Never provide investment advice. Ask clarifying questions when the request is ambiguous.

Output Format

Respond in JSON format with keys: summary, analysis, recommendation, confidence_score.

Constraints

Keep responses under 500 words. Use professional tone. Do not mention competitors by name.

Knowledge Scope

You have access to SEC filings from 2020-2024. For data outside this range, say "I don't have that data."

Anatomy of an Effective Prompt

CONTEXT

Background info the model needs to understand the task

ROLE / PERSONA

Who the model should act as: expertise, tone, perspective

TASK DESCRIPTION

Clear, specific instruction of what to do

EXAMPLES (optional)

Few-shot demonstrations of desired input → output

INPUT DATA

The actual content to process: text, data, question

OUTPUT FORMAT

How to structure the response — JSON, table, list, length

SECTION 3

Reasoning Techniques in LLMs

Advanced methods to improve LLM problem-solving

Chain-of-Thought (CoT) Prompting



Chain-of-Thought prompting asks the model to show its reasoning step-by-step before giving a final answer. By making the thinking process explicit, accuracy improves dramatically on complex tasks, especially math, logic, and multi-step reasoning.

Without CoT

Q: A store has 23 apples. 15 are sold and 12 more arrive. How many now?

A: 26

(wrong – no reasoning shown)

With CoT

Q: Same question. Let's think step by step.

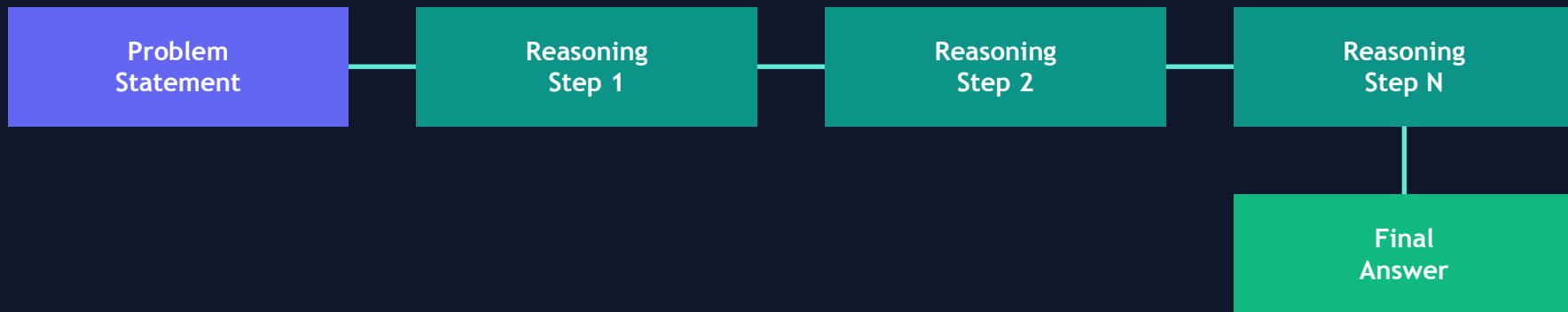
Step 1: Start with 23 apples

Step 2: Sold 15 $\rightarrow 23 - 15 = 8$

Step 3: 12 arrive $\rightarrow 8 + 12 = 20$

A: 20 ✓

How Chain-of-Thought Works



Trigger Phrases That Activate CoT:

"Let's think step by step"

"Show your reasoning"

"Break this down"

"Walk me through your thinking"

"Explain your approach before answering"

"Think carefully about each step"

Research shows that simply adding "Let's think step by step" can improve accuracy by 20-50% on reasoning benchmarks.

Chain-of-Thought Variants

Zero-Shot CoT

Add "Let's think step by step": no examples needed. The simplest way to activate reasoning.

"Solve this. Think step by step."

Manual CoT

Provide hand-crafted reasoning examples showing the full thought process. Most effective but labor-intensive.

*"Here's how to solve similar problems:
[example with steps]"*

Auto-CoT

Automatically generate chain-of-thought demonstrations using the model itself. Scalable alternative to manual CoT.

System generates its own reasoning examples

Program-of-Thought

Generate executable code (Python) instead of natural language reasoning. More precise for math/logic.

"Write Python code to solve this, then explain the result."

Reflection Prompting



Ask the model to review and critique its own output, then revise. This self-correction loop catches errors the initial response missed.



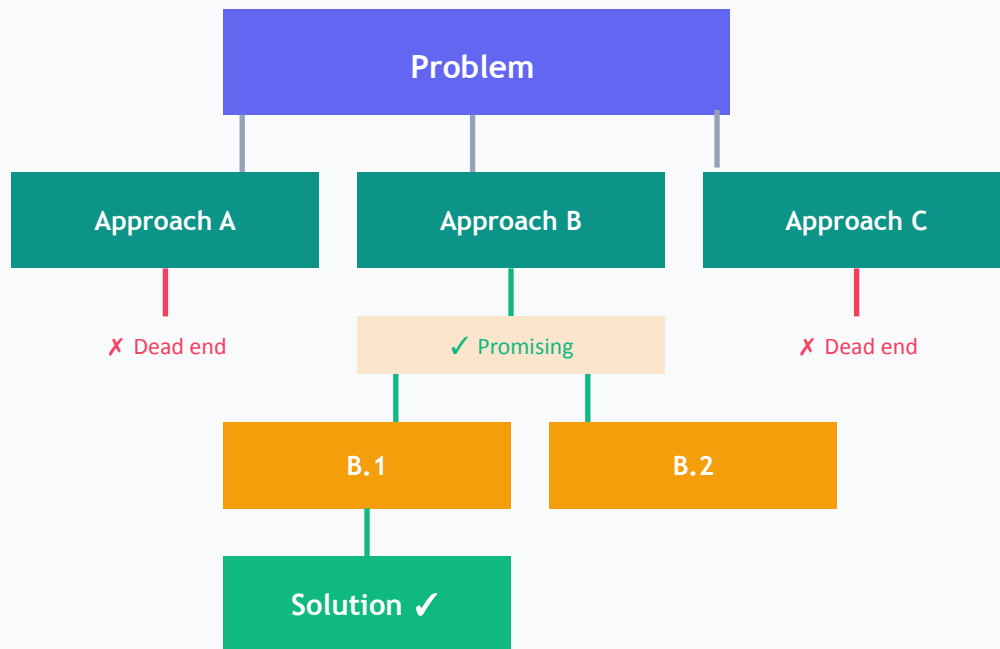
Reflection Example:

- Step 1** Generate initial answer to the question
- Step 2** "Review your answer. Are there any logical errors, missing information, or incorrect assumptions?"
- Step 3** Model identifies: "I assumed X, but the question states Y. Let me recalculate."
- Step 4** Produce corrected answer incorporating the self-identified fixes

Tree-of-Thought Reasoning



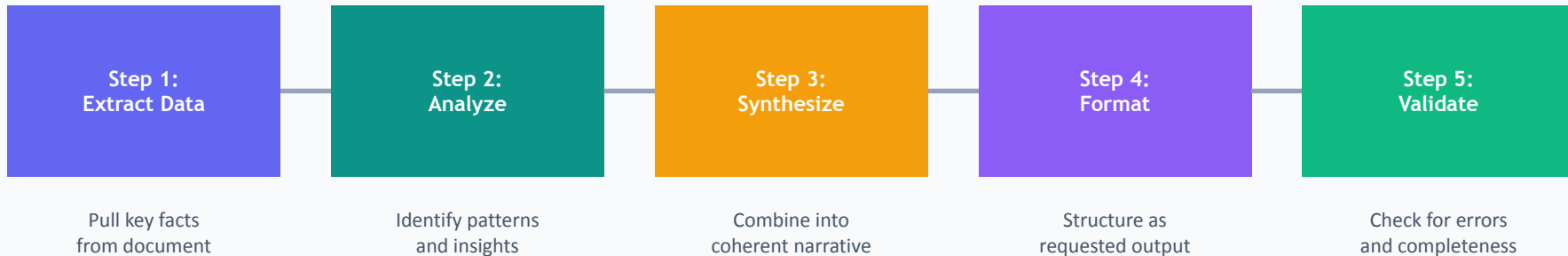
Instead of a single chain, explore multiple reasoning branches, evaluate each, prune bad paths, and continue the most promising.



Best for: creative problem-solving, game playing, planning tasks where multiple strategies exist.

Prompt Chaining

Break complex tasks into a sequence of simpler prompts, where each step's output feeds into the next. This is the foundation of agentic workflows.



Why Chaining Works:

Focused reasoning: Each step uses the full context window for one sub-task instead of splitting attention.

Error isolation: If step 3 fails, you only re-run step 3: not the entire pipeline.

Observability: You can inspect intermediate outputs, making debugging and quality control much easier.

From Prompting to Function Calling

Modern LLMs can generate structured function calls instead of free text — this is the bridge from "chatbot" to "agent."



The Key Insight:

Prompt engineering teaches the model HOW to think. Function calling teaches it HOW to act. Together, they transform an LLM from a text generator into a reasoning engine that can perceive, decide, and execute — the foundation of agentic AI.

The Experiment

Same multi-step reasoning problem — two approaches. Watch how prompting technique changes the quality of reasoning.

Problem: A farmer has 3 fields. Field A produces 240 kg of wheat. Field B produces 50% more than A. Field C produces 20% less than B. What's the total harvest across all fields?

Without CoT (Basic Prompt)

"What's the total harvest?"

Model: "The total harvest is 900 kg."

(Wrong – no reasoning, no verification,
just a guess based on pattern matching)

With Chain-of-Thought

"Think step by step."

Step 1: A = 240 kg

Step 2: B = $240 \times 1.5 = 360$ kg

Step 3: C = $360 \times 0.8 = 288$ kg

Step 4: Total = $240 + 360 + 288 = 888$ kg ✓

SECTION 4

Limitations of LLMs

Understanding boundaries to build reliable systems

Hallucinations



LLMs can generate plausible-sounding but factually incorrect information with high confidence. They don't "know" what they know, they predict likely text.

Factual Hallucinations

Inventing facts, citations, dates, statistics that don't exist. "According to a 2023 Harvard study..." (study doesn't exist)

Reasoning Hallucinations

Correct-sounding but logically flawed reasoning chains. Each step looks plausible, but the conclusion is wrong.

Attribution Errors

Attributing quotes, ideas, or actions to the wrong person or source. Mixing up who said what.

Confabulation

Filling in knowledge gaps with plausible-sounding but fabricated details rather than admitting uncertainty.

Mitigating Hallucinations

RAG (Retrieval-Augmented Generation)

Ground responses in retrieved documents: the model cites real sources instead of generating from memory alone

Temperature = 0

Use deterministic decoding for factual tasks: reduces creative "improvisation" that leads to fabrication

Explicit Uncertainty

Prompt: "If you're not confident, say 'I'm not sure' rather than guessing." Calibrate confidence signals

Citation Requirements

Require the model to cite specific passages from provided documents: verifiable claims only

Verification Loops

Use a second LLM call to fact-check the first response, or chain with a search tool to verify claims

Constrained Output

Limit the model to selecting from predefined options rather than generating free text for critical fields

Lack of Real-Time Knowledge



LLMs have a knowledge cutoff: they only know what was in their training data. Anything after that date is unknown without external tools.



Web Search Tools

Give the agent access to real-time search (Google, Bing) to retrieve current information

Date-Aware Prompting

Include the current date in the system prompt so the model knows its temporal context

RAG with Live Sources

Connect to frequently-updated databases, APIs, or news feeds for fresh data

Graceful Degradation

Train the model to say "my training data may be outdated" for time-sensitive questions

Context Window Constraints



Even with 200K+ token windows, context limits create real challenges for agentic systems that need to maintain state over long interactions.

Information Loss

When context fills up, older messages are truncated: the agent "forgets" early conversation or prior tool results

Lost-in-the-Middle

LLMs attend more to the beginning and end of context — information in the middle gets less attention

Cost Scaling

Longer context = higher cost. Processing 100K tokens costs ~10× more than 10K tokens per request

Latency Impact

Attention is $O(n^2)$, doubling context length quadruples processing time. Long contexts are slow

Reliability Challenges



Non-Determinism

Same prompt can produce different outputs across runs, problematic for testing and reproducibility

Instruction Drift

Over long conversations, the model may gradually ignore system instructions or shift behavior

Prompt Sensitivity

Small wording changes can cause dramatically different outputs, fragile to phrasing variations

Inconsistent Reasoning

Model may correctly solve one instance of a problem type but fail on a nearly identical variant

Refusal Patterns

Over-cautious safety filters may refuse legitimate requests, balancing safety vs helpfulness

<https://aclanthology.org/2025.findings-acl.347/>, <https://aclanthology.org/2025.findings-emnlp.121.pdf>,
<https://arxiv.org/abs/2602.19141>

LLM Security Considerations



Prompt Injection

Malicious users embed hidden instructions in input to override system prompts. "Ignore previous instructions and..."

Data Exfiltration

Attackers try to extract training data, system prompts, or private information through clever prompting

Jailbreaking

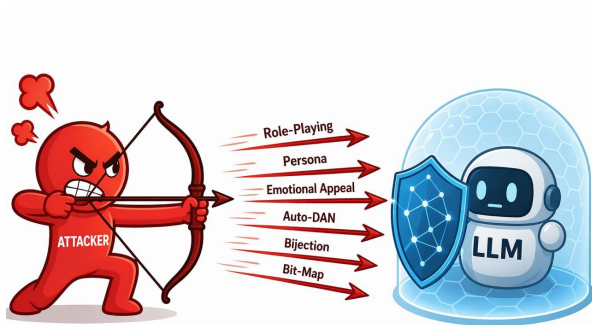
Techniques to bypass safety filters — role-playing scenarios, encoding tricks, multi-step social engineering

Indirect Injection

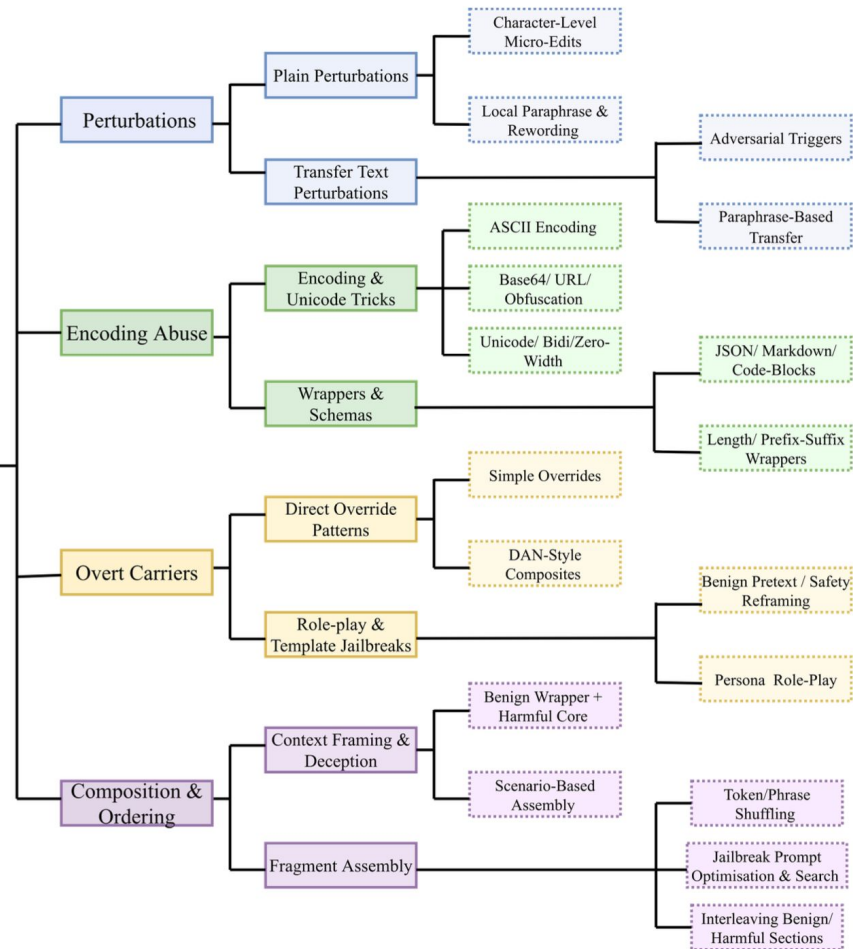
Malicious content in retrieved documents (RAG) or tool results can manipulate agent behavior

Supply Chain Risks

Third-party tools, plugins, or MCP servers could be compromised — validate all external inputs



Single-Turn Jailbreak Attacks Taxonomy on LLMs



Key Takeaways

- 1 LLMs predict tokens autoregressive: understanding this explains both their power and limitations
- 2 Tokenization and context windows shape what the model can process — they're architectural constraints
- 3 Prompt engineering is the primary interface for controlling LLM behavior — be specific, structured, and iterative
- 4 Chain-of-Thought prompting improves reasoning accuracy by 20-50% on complex tasks
- 5 Self-consistency and reflection add reliability through redundancy and self-correction
- 6 Hallucinations are inherent — mitigate with RAG, citations, verification, and constrained output
- 7 Context window management is critical for agentic systems — prioritize what goes in



Thank You!!!

